

UNIVERSIDAD DE LA COSTA (CUC)

RÚBRICA – SISTEMA BANCARIO

Proyecto de Programación en Python

Estudiantes: Cristian Ledezma
Carlos Monroy

Profesor: Michael Puche

Asignatura: Programación Orientada a Objetos

Tema: Sistema Bancario

Norma: Formato APA

Fecha: 28/05/2026

Introduccion

El proyecto consiste en el desarrollo de un sistema bancario en Python utilizando Programación Orientada a Objetos (POO) y una interfaz gráfica creada con Tkinter.

La aplicación permite gestionar clientes, cuentas, tarjetas, créditos, ahorros y bolsillos virtuales, simulando el funcionamiento básico de un banco digital.

El sistema fue diseñado con una arquitectura modular basada en clases, permitiendo una mejor organización y mantenimiento del código.

Cada módulo representa una entidad bancaria, como clientes, tarjetas, créditos y ahorros, integrando funcionalidades financieras reales como depósitos, retiros, pagos y solicitudes de crédito.

Además, el proyecto implementa persistencia de datos mediante archivos JSON, permitiendo guardar la información incluso después de cerrar la aplicación.

La interfaz gráfica facilita la interacción del usuario mediante menús intuitivos y validaciones de seguridad.

Entre las funcionalidades principales se encuentran:

- Registro e inicio de sesión de usuarios.
- Gestión de saldo y movimientos financieros.
- Administración de tarjetas débito y crédito.
- Creación de bolsillos virtuales.
- Sistema de ahorro con cálculo de intereses.
- Solicitud y pago de créditos.
- Validación y seguridad de datos.

El proyecto demuestra la aplicación práctica de conceptos fundamentales de programación como encapsulamiento, modularidad, manejo de archivos, validación de datos y diseño de interfaces gráficas.

Explicación Completa del Proyecto Banco

Este documento explica el funcionamiento general del proyecto “Banco”, describiendo la estructura del sistema, las clases implementadas, las funciones principales y la lógica usada para manejar usuarios, tarjetas, créditos, bolsillos y ahorros.

1. Estructura General del Proyecto

Archivo	Función dentro del proyecto
Main.py	Contiene la lógica principal del sistema, validaciones y manejo de datos
gui.py	Interfaz gráfica realizada con Tkinter.
Cliente.py	Define la clase Cliente.
Tarjeta.py	Maneja tarjetas débito y crédito.
Credito.py	Administra préstamos y pagos.
Bolsillo.py	Sistema de bolsillos para separar dinero.
Ahorro.py	Sistema de ahorro con interés diario.
Archivos JSON	Guardan la información de manera persistente.

2. Explicación de Main.py

Main.py es el núcleo lógico del proyecto. Este archivo controla:

- La carga y guardado de datos.
- Las validaciones de usuarios.
- La creación y recuperación de clientes.
- El manejo de archivos JSON.
- La comunicación entre las clases y la interfaz.

El programa usa diccionarios y archivos JSON para almacenar información de clientes, tarjetas, créditos, bolsillos y ahorros.

Funciones principales

- **ensure_data_files()**: Verifica si existen los archivos JSON y los crea automáticamente si faltan.
- **load_json()**: Carga información desde un archivo JSON.
- **save_json()**: Guarda información dentro de un archivo JSON.
- **load_all_data()**: Carga todos los datos del sistema en memoria.

- **cargar_cliente():** Reconstruye un cliente junto con sus tarjetas, créditos, bolsillos y ahorros.
- **guardar_cliente():** Guarda toda la información actualizada de un cliente.
- **obtener_ahorro():** Garantiza que cada cliente tenga un ahorro principal.

3. Clase Cliente

La clase Cliente representa a cada usuario del banco. Contiene toda la información principal del usuario:

- Nombre y apellidos.
- Documento de identidad.
- Teléfono y correo.
- Contraseña.
- Saldo disponible.
- Cupo de crédito.
- Listas de tarjetas, créditos, bolsillos y ahorros.

Además, incluye métodos como:

- **to_dict():** convierte el objeto a formato JSON.
- **from_dict():** reconstruye el cliente desde JSON.
- **verificar_contrasena():** valida el login.
- **consultar_saldo_total():** suma todos los fondos del usuario.

4. Clase Tarjeta

La clase Tarjeta maneja tanto tarjetas débito como crédito. Cada tarjeta posee:

- Número de tarjeta.
- Tipo de tarjeta.
- Código CVC.
- Saldo utilizado.
- Estado activa/bloqueada.

Funciones importantes

- **activar() y bloquear():** cambian el estado de la tarjeta.
- **puede_pagar():** valida si existe saldo suficiente.
- **realizar_pago():** descuenta dinero o usa el cupo de crédito.

La lógica cambia dependiendo del tipo de tarjeta:

- **Débito** → usa dinero disponible del cliente.
- **Crédito** → usa el cupo de crédito disponible.

5. Clase Credito

La clase Credito representa préstamos bancarios. Cuando se crea un crédito:

- Se almacena el capital solicitado.
- Se calcula una deuda con interés del 8%.
- Se divide el pago en cuotas.

Funciones principales

- **cuota()**: calcula el valor de cada cuota.
- **pagar()**: permite abonar dinero a la deuda.
- **consultar_deuda()**: devuelve el saldo pendiente.

El sistema evita pagos inválidos o superiores al dinero disponible.

6. Clase Bolsillo

La clase Bolsillo funciona como una división interna del dinero. Permite separar fondos para diferentes objetivos.

Ejemplos

- Viajes.
- Estudios.
- Compras.

Funciones

- **depositar()**: mueve dinero desde el saldo principal al bolsillo.
- **retirar()**: devuelve dinero al saldo principal.
- **consultar_saldo()**: muestra cuánto dinero hay guardado.

7. Clase Ahorro

La clase Ahorro implementa una cuenta de ahorro con interés diario. El sistema registra la fecha del último cálculo y aplica intereses automáticamente.

Características

- Tasa de interés del 5%.
- Cálculo automático de crecimiento diario.
- Depósitos y retiros.
- Consulta del saldo actualizado.

Funciones importantes

- **actualizar_interes()**: calcula intereses según el tiempo transcurrido.
- **incremento_diario()**: estima cuánto crecerá el ahorro diariamente.

8. Interfaz Gráfica (gui.py)

La interfaz gráfica fue desarrollada usando Tkinter. El archivo gui.py permite que el usuario interactúe con el sistema sin necesidad de usar la consola.

La interfaz incluye:

- Registro de usuarios.
- Inicio de sesión.
- Menús interactivos.
- Gestión de tarjetas.
- Gestión de créditos.
- Gestión de bolsillos.
- Gestión de ahorros.

El sistema usa ventanas, botones, labels y cuadros de texto para facilitar la experiencia del usuario.

9. Persistencia de Datos con JSON

El proyecto utiliza archivos JSON para guardar información permanentemente. Esto permite que los datos no se pierdan al cerrar el programa.

Archivos utilizados

- clientes.json
- tarjetas.json
- creditos.json
- bolsillos.json

- ahorros.json

Cada archivo almacena información específica del sistema.

10. Flujo General del Sistema

1. El usuario se registra o inicia sesión.
2. El sistema carga los datos desde JSON.
3. Se reconstruye el objeto Cliente.
4. El usuario puede operar con tarjetas, créditos, bolsillos y ahorros.
5. Cada operación actualiza los datos en memoria.
6. Los cambios se guardan automáticamente en JSON.

11. Conclusión

Este proyecto simula un sistema bancario completo orientado a objetos. Se implementaron múltiples conceptos de programación:

- Programación orientada a objetos.
- Persistencia de datos.
- Interfaces gráficas.
- Validación de datos.
- Manejo de archivos.
- Modularización del código.

La separación en clases permite que el proyecto sea organizado, escalable y fácil de mantener.